



МИНОБРНАУКИ РОССИИ

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

**Институт информационных технологий (ИИТ)
Кафедра прикладной математики (ПМ)**

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №5
по дисциплине «Технологии и инструментарий машинного обучения»

Студент группы *ИНБО-20-23. Боргачев Т.М.*

(подпись)

Преподаватель *Трушин СМ*

(подпись)

Москва 2025 г.

СОДЕРЖАНИЕ

1 ПРАКТИЧЕСКАЯ РАБОТА №5	3
1.1 Задание 1. Дерево решений	3
1.2 Задание 2. Случайный лес	13
1.3 Задание 3. Градиентный бустинг	15
1.4 Задание 4. Стэкинг	20
ВЫВОД	26

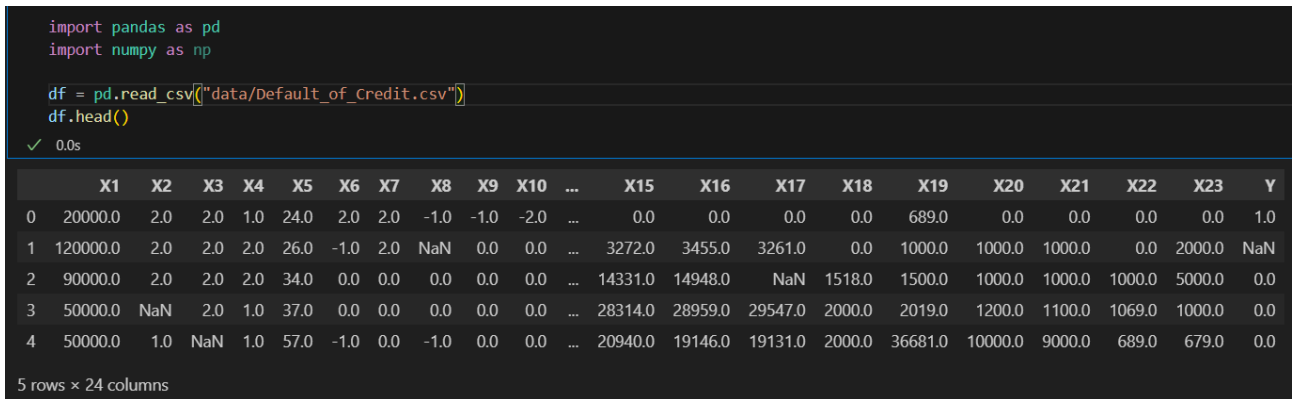
1 ПРАКТИЧЕСКАЯ РАБОТА №5

1.1 Задание 1. Дерево решений

Ссылка на Google colab с выполненной работой:
https://colab.research.google.com/drive/1npZrD_c4caWTBPmQvBgFziqR2PGJari?usp=sharing.

В качестве данных будем использовать датасет «Default_of_Credit.csv» — исследование, направленное на изучение случаев неплатежей клиентов на Тайване и сравнение точности прогнозирования вероятности дефолта среди шести методов интеллектуального анализа данных.

Обзор данных в Python представлен на Рисунке 1.



```
import pandas as pd
import numpy as np

df = pd.read_csv("data/Default_of_Credit.csv")
df.head()
```

✓ 0.0s

	X1	X2	X3	X4	X5	X6	X7	X8	X9	X10	...	X15	X16	X17	X18	X19	X20	X21	X22	X23	Y
0	20000.0	2.0	2.0	1.0	24.0	2.0	2.0	-1.0	-1.0	-2.0	...	0.0	0.0	0.0	0.0	689.0	0.0	0.0	0.0	0.0	1.0
1	120000.0	2.0	2.0	2.0	26.0	-1.0	2.0	NaN	0.0	0.0	...	3272.0	3455.0	3261.0	0.0	1000.0	1000.0	1000.0	0.0	2000.0	NaN
2	90000.0	2.0	2.0	2.0	34.0	0.0	0.0	0.0	0.0	0.0	...	14331.0	14948.0	NaN	1518.0	1500.0	1000.0	1000.0	1000.0	5000.0	0.0
3	50000.0	NaN	2.0	1.0	37.0	0.0	0.0	0.0	0.0	0.0	...	28314.0	28959.0	29547.0	2000.0	2019.0	1200.0	1100.0	1069.0	1000.0	0.0
4	50000.0	1.0	NaN	1.0	57.0	-1.0	0.0	-1.0	0.0	0.0	...	20940.0	19146.0	19131.0	2000.0	36681.0	10000.0	9000.0	689.0	679.0	0.0

5 rows × 24 columns

Рисунок 1 — Первые строки датасета

Размер, дубликаты и пропуски данных представлены на Рисунке 2.

```
print("Размер:", df.shape)
print("\nПропуски:", df.isnull().sum())
print("\nДубликаты:", df.duplicated().sum())
```

✓ 0.0s

Размер: (30034, 24)

Пропуски:	X1	1520
X2	1495	
X3	1497	
X4	1487	
X5	1495	
X6	1469	
X7	1554	
X8	1493	
X9	1519	
X10	1515	
X11	1517	
X12	1504	
X13	1500	
X14	1519	
X15	1470	
X16	1508	
X17	1525	
X18	1519	
X19	1440	
X20	1478	
X21	1545	
X22	1457	
X23	1512	
Y	1501	

dtype: int64

Дубликаты: 40

Рисунок 2 — Информация о данных

Загрузка данных в Logiном представлена на Рисунке 3.

Default_Credit • Быстрый просмотр																			
Набор данных																			
#	X1	X2	X3	X4	X5	X6	X7	X8	X9	X10	X11	X12	X13	X14	X15	X16	X17	X18	X19
1	20 000,00	2,00	2,00	1,00	24,00	2,00	2,00	-1,00	-1,00	-2,00	-2,00		3 102,00	689,00	0,00	0,00	0,00	0,00	0,00
2	120 000,00	2,00	2,00	2,00	26,00	-1,00	2,00		0,00	0,00	2,00		1 725,00	2 682,00	3 272,00	3 455,00	3 261,00	0,00	
3	90 000,00	2,00	2,00	2,00	34,00	0,00	0,00	0,00	0,00	0,00	0,00	29 239,00	14 027,00	13 559,00	14 331,00	14 948,00		1 518,00	
4	50 000,00		2,00	1,00	37,00	0,00	0,00	0,00	0,00	0,00	0,00	46 990,00	48 233,00	49 291,00	28 314,00	28 959,00	29 547,00	2 000,00	
5	50 000,00	1,00		1,00	57,00	-1,00	0,00	-1,00	0,00	0,00	0,00	8 617,00	5 670,00	35 835,00	20 940,00	19 146,00	19 131,00	2 000,00	3
6	50 000,00	1,00	1,00	2,00	37,00	0,00	0,00	0,00	0,00	0,00		64 400,00	57 069,00	57 608,00	19 394,00		20 024,00		
7	500 000,00	1,00	1,00	2,00	29,00	0,00	0,00	0,00	0,00	0,00	0,00	367 965,00	412 023,00		542 653,00	483 003,00	473 944,00	55 000,00	4
8	100 000,00	2,00	2,00	2,00	23,00	0,00		-1,00	0,00	0,00	-1,00	11 876,00	380,00	601,00	221,00	-159,00	567,00	380,00	
9	140 000,00	2,00	3,00	1,00	28,00	0,00	0,00	2,00	0,00	0,00	0,00	11 285,00	14 096,00	12 108,00	12 211,00	11 793,00	3 719,00	3 329,00	
10	20 000,00	1,00	3,00	2,00	35,00	-2,00	-2,00	-2,00	-2,00	-1,00	0,00	0,00	0,00	0,00	0,00	13 007,00	13 912,00	0,00	

Рисунок 3 — Загрузка данных в LogiNot

Информация о пропусках и выбросах в LogiNot представлена на Рисунке 4.

Сводка										Показатели		XLS	
№	Тип	Метка	Вид	Проблемы	Виды проблем				Характеристика набора данных		Значение		
9	X10			91,10%	Пропуски - 5,04% (1 515)	Экстремальные - 0,21% (62)	Выбросы - 0,32% (96)	Отриц.	Метод определения нетипичных значений	Стандартное откл...			
10	X11			90,63%	Пропуски - 5,05% (1 517)	Экстремальные - 0,21% (63)	Выбросы - 0,19% (58)	Отриц.	Столбцов	24			
8	X9			89,41%	Пропуски - 5,06% (1 519)	Экстремальные - 0,18% (55)	Выбросы - 0,33% (99)	Отриц.	Строк	30 034			
7	X8			87,03%	Пропуски - 4,97% (1 493)	Экстремальные - 0,17% (52)	Выбросы - 0,31% (94)	Отриц.	Заполненных полей	0,00%			
6	X7			86,43%	Пропуски - 5,17% (1 554)	Экстремальные - 0,10% (31)	Выбросы - 0,39% (117)	Отриц.	Полных записей	29,34%			
23	Y			78,98%	Пропуски - 5,00% (1 501)	Нули - 73,99% (22 221)			Пригодных столбцов	24 из 24			
5	X6			78,77%	Пропуски - 4,89% (1 469)	Экстремальные - 0,12% (37)	Выбросы - 0,33% (99)	Отриц.	Индекс EPV	1 251,42			
22	X23			29,11%	Пропуски - 5,03% (1 512)	Экстремальные - 0,81% (242)	Выбросы - 0,57% (170)	Нуль					
21	X22			27,45%	Пропуски - 4,85% (1 457)	Экстремальные - 0,73% (219)	Выбросы - 0,58% (175)	Нуль					
20	X21			26,59%	Пропуски - 5,14% (1 545)	Экстремальные - 0,66% (198)	Выбросы - 0,59% (178)	Нуль					
19	X20			25,04%	Пропуски - 4,92% (1 478)	Экстремальные - 0,65% (196)	Выбросы - 0,50% (150)	Нуль					
18	X19			23,09%	Пропуски - 4,79% (1 440)	Экстремальные - 0,54% (161)	Выбросы - 0,54% (161)	Нуль					
17	X18			22,87%	Пропуски - 5,06% (1 519)	Экстремальные - 0,62% (185)	Выбросы - 0,63% (189)	Нуль					
16	X17			22,07%	Пропуски - 5,08% (1 525)	Экстремальные - 0,45% (135)	Выбросы - 1,61% (484)	Отриц.					
15	X16			20,20%	Пропуски - 5,02% (1 508)	Экстремальные - 0,46% (139)	Выбросы - 1,57% (472)	Отриц.					
14	X15			19,30%	Пропуски - 4,89% (1 470)	Экстремальные - 0,47% (140)	Выбросы - 1,68% (505)	Отриц.					
13	X14			18,29%	Пропуски - 5,06% (1 519)	Экстремальные - 0,43% (129)	Выбросы - 1,66% (499)	Отриц.					
12	X13			17,19%	Пропуски - 4,99% (1 500)	Экстремальные - 0,44% (131)	Выбросы - 1,68% (506)	Отриц.					
11	X12			15,39%	Пропуски - 5,01% (1 504)	Экстремальные - 0,46% (138)	Выбросы - 1,72% (516)	Отриц.					
2	X3			6,07%	Пропуски - 4,98% (1 497)	Экстремальные - 0,16% (48)	Выбросы - 0,88% (264)	Нуль					
0	X1			5,48%	Пропуски - 5,06% (1 520)	Экстремальные - 0,00% (1)	Выбросы - 0,42% (125)						
4	X5			5,42%	Пропуски - 4,98% (1 495)	Выбросы - 0,44% (133)							
3	X4			5,12%	Пропуски - 4,95% (1 487)	Нули - 0,17% (51)							
1	X2			4,98%	Пропуски - 4,98% (1 495)								

Рисунок 4 — Информация о пропусках и выбросах

Предобработка данных в LogiNot представлена на Рисунке 5.

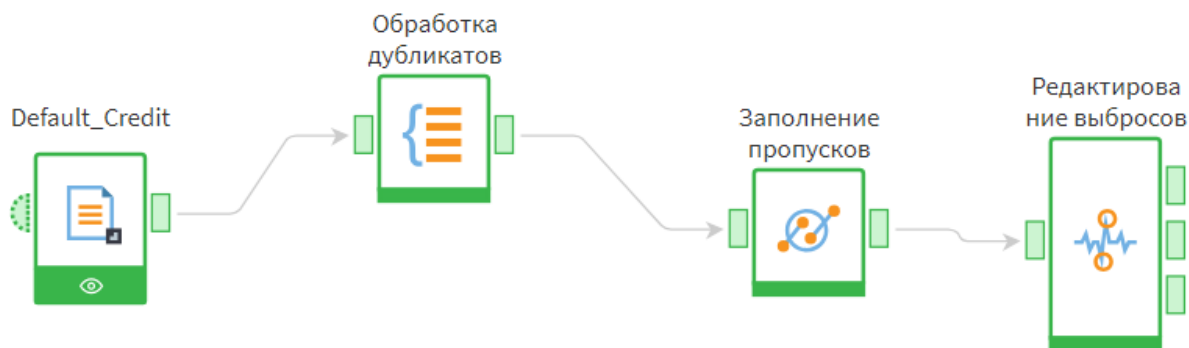


Рисунок 5 — Предобработка данных LogiNot

Код для предобработки данных в Python представлен в Листинге 1.

Листинг 1 — Предобработка Python

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer

df = df.drop_duplicates()

df = df.dropna(subset=['Y'])

X = df.drop(columns=['Y'])
y = df['Y'].astype(int)

for col in X.columns[:10]:
    uniq = X[col].dropna().unique()
    print(f"{col}: {sorted(uniq)[:10]} ({len(uniq)} уникальных)")

num_cols = ['X1', 'X5', 'X15', 'X16', 'X17', 'X18', 'X19', 'X20',
            'X21', 'X22', 'X23']
cat_cols = [col for col in X.columns if col not in num_cols]

for col in cat_cols:
    X[col] = X[col].replace(-1, np.nan)

num_imputer = SimpleImputer(strategy='median')
cat_imputer = SimpleImputer(strategy='most_frequent')

X_num = pd.DataFrame(num_imputer.fit_transform(X[num_cols]),
                     columns=num_cols)
X_cat = pd.DataFrame(cat_imputer.fit_transform(X[cat_cols]),
                     columns=cat_cols)

X_cat_encoded = pd.get_dummies(X_cat, columns=cat_cols,
                               drop_first=True)

scaler = StandardScaler()
X_num_scaled = pd.DataFrame(scaler.fit_transform(X_num),
                             columns=num_cols)

X_processed = pd.concat([X_num_scaled, X_cat_encoded], axis=1)

X_train, X_test, y_train, y_test = train_test_split(
    X_processed, y, test_size=0.2, stratify=y, random_state=42
)

print("Train shape:", X_train.shape)
print("Test shape:", X_test.shape)
print("Распределение классов в train:", np.bincount(y_train))
```

Результат предобработки представлен на Рисунке 6.

```

X1: [np.float64(10000.0), np.float64(16000.0), np.float64(20000.0), np.float64(30000.0), np.float64(40000.0), np.float64(50000.0), np.float64(60000.0), np.float64(70000.0), np.float64(80000.0)] (2 уникальных)
X2: [np.float64(1.0), np.float64(2.0)] (2 уникальных)
X3: [np.float64(0.0), np.float64(1.0), np.float64(2.0), np.float64(3.0), np.float64(4.0), np.float64(5.0), np.float64(6.0)] (7 уникальных)
X4: [np.float64(0.0), np.float64(1.0), np.float64(2.0), np.float64(3.0)] (4 уникальных)
X5: [np.float64(21.0), np.float64(22.0), np.float64(23.0), np.float64(24.0), np.float64(25.0), np.float64(26.0), np.float64(27.0), np.float64(28.0), np.float64(29.0), np.float64(30.0)] (10 уникальных)
X6: [np.float64(-2.0), np.float64(-1.0), np.float64(0.0), np.float64(1.0), np.float64(2.0), np.float64(3.0), np.float64(4.0), np.float64(5.0), np.float64(6.0), np.float64(7.0)] (11 уникальных)
X7: [np.float64(-2.0), np.float64(-1.0), np.float64(0.0), np.float64(1.0), np.float64(2.0), np.float64(3.0), np.float64(4.0), np.float64(5.0), np.float64(6.0), np.float64(7.0)] (11 уникальных)
X8: [np.float64(-2.0), np.float64(-1.0), np.float64(0.0), np.float64(1.0), np.float64(2.0), np.float64(3.0), np.float64(4.0), np.float64(5.0), np.float64(6.0), np.float64(7.0)] (11 уникальных)
X9: [np.float64(-2.0), np.float64(-1.0), np.float64(0.0), np.float64(1.0), np.float64(2.0), np.float64(3.0), np.float64(4.0), np.float64(5.0), np.float64(6.0), np.float64(7.0)] (11 уникальных)
X10: [np.float64(-2.0), np.float64(-1.0), np.float64(0.0), np.float64(1.0), np.float64(2.0), np.float64(3.0), np.float64(4.0), np.float64(5.0), np.float64(6.0), np.float64(7.0), np.float64(8.0)] (12 уникальных)
Train shape: (22794, 61484)
Test shape: (5699, 61484)
Распределение классов в train: [17751 5043]

```

Рисунок 6 — Результат предобработки

Реализация дерева решений в Logiom представлена на Рисунке 7.

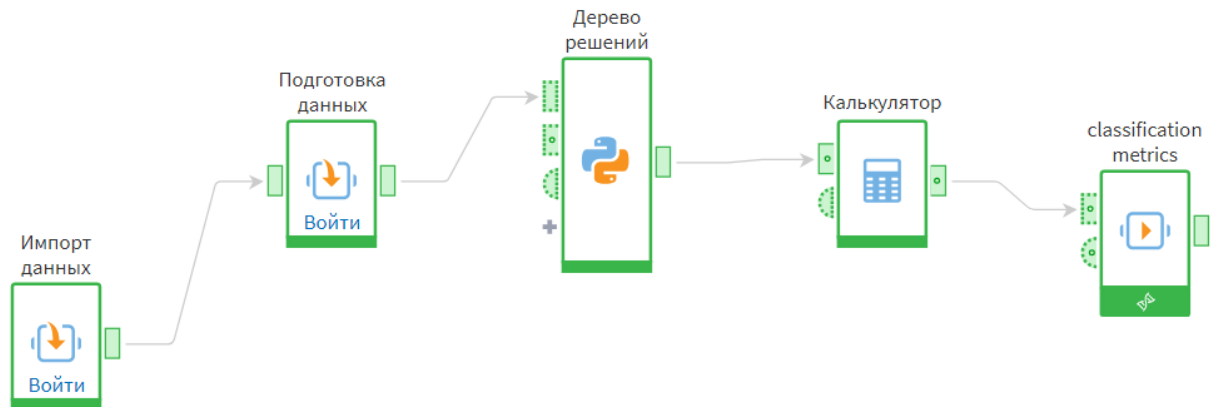


Рисунок 7 — Дерево решений Logiom

Код для дерева решений в Python представлен в Листинге 2.

Листинг 2 — Дерево решений Python

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report, roc_auc_score,
confusion_matrix
import time

models = {
    'Gini': DecisionTreeClassifier(criterion='gini',
random_state=42),
    'Entropy': DecisionTreeClassifier(criterion='entropy',
random_state=42)
}

results = {}

for name, model in models.items():
    start = time.time()
    model.fit(X_train, y_train)
    train_time = time.time() - start

```

Продолжение Листинга 2

```
y_pred = model.predict(X_test)
y_proba = model.predict_proba(X_test)[:, 1]
results[name] = {
    'model': model,
    'time': train_time,
    'pred': y_pred,
    'proba': y_proba,
    'report': classification_report(y_test, y_pred,
output_dict=True),
    'auc': roc_auc_score(y_test, y_proba)
}

print(f"Accuracy: {results[name]['report']['accuracy']:.4f}")
print(f"F1: {results[name]['report']['macro avg']['f1-
score']:.4f}")
print(f"AUC: {results[name]['auc']:.4f}")
print(f"Время обучения: {train_time:.3f} сек")
```

Метрики дерева решений в Logiном представлены на Рисунке 8.

Выходной набор данных		
#	ab Метрика	90 Значение
1	precision	0,37
2	recall	0,40
3	f1_score	0,38
4	fbeta_score	0,38
5	accuracy	0,73
6	tn	3 878,00
7	fp	867,00
8	fn	759,00
9	tp	503,00

Рисунок 8 —Метрики дерева решений Logiном

Метрики дерева решений в Python представлены на Рисунке 9.

```

=== Gini ===
Accuracy: 0.7724
F1: 0.6396
AUC: 0.6296
Время обучения: 88.823 сек

=== Entropy ===
Accuracy: 0.7408
F1: 0.6211
AUC: 0.6201
Время обучения: 32.669 сек

```

Рисунок 9 — Метрики дерева решений Python

Код для оптимизации гиперпараметров и визуализации представлен в Листинге 3.

Листинг 3 — Подбор гиперпараметров и визуализация

```

from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split,
GridSearchCV, RandomizedSearchCV
from sklearn.metrics import classification_report, roc_auc_score,
roc_curve
import matplotlib.pyplot as plt

X_train_small, _, Y_train_small, _ = train_test_split(
    X_train, y_train,
    train_size=0.2,
    stratify=y_train,
    random_state=42
)

param_dist = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [5, 10, 15],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 3, 5]
}

dt = DecisionTreeClassifier(random_state=42)
grid = RandomizedSearchCV(
    dt,
    param_distributions=param_dist,
    n_iter=15,
    cv=3,
    scoring='f1',
    n_jobs=-1,
    random_state=42,

```

Продолжение Листинга 3

```
        error_score='raise'
    )

    try:
        grid.fit(X_train_small, Y_train_small)
    except Exception as e:
        print(f"Ошибка при поиске гиперпараметров: {str(e)}")
        print("Используем модель по умолчанию")
        grid.best_estimator_ = dt.fit(X_train, y_train)

best_dt = grid.best_estimator_.fit(X_train, y_train)

Y_pred_best = best_dt.predict(X_test)
Y_proba_best = best_dt.predict_proba(X_test)[:, 1]

print("Лучшие параметры:", grid.best_params_)
print(classification_report(y_test, Y_pred_best))
print("AUC:", roc_auc_score(y_test, Y_proba_best))


plt.figure(figsize=(10, 6))
fpr, tpr, _ = roc_curve(y_test, Y_proba_best)
plt.plot(fpr, tpr,
         label=f"Decision Tree (AUC={roc_auc_score(y_test, Y_proba_best):.3f})",
         linewidth=2)
plt.plot([0, 1], [0, 1], 'k--', alpha=0.7)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend(loc="lower right")
plt.grid(alpha=0.3)
plt.show()


plt.figure(figsize=(20, 15))
plot_tree(
    best_dt,
    max_depth=20,
    feature_names=X_train.columns.tolist(),
    class_names=['Class 0', 'Class 1'],
    filled=True,
    rounded=True,
    fontsize=10
)
plt.show()
```

Подобранные гиперпараметры представлены на Рисунке 10.

```

лучшие параметры: {'min_samples_split': 10, 'min_samples_leaf': 3, 'max_depth': 5, 'criterion': 'entropy'}
precision    recall  f1-score   support

      0       0.83      0.95      0.89      4438
      1       0.64      0.33      0.43      1261

 accuracy          0.81      5699
 macro avg       0.74      0.64      0.66      5699
weighted avg       0.79      0.81      0.79      5699

AUC: 0.7363702348579907
Графики сохранены: roc_curve.png, decision_tree.png

```

Рисунок 10 — Оптимизированные гиперпараметры

ROC кривая представлена на Рисунке 11.

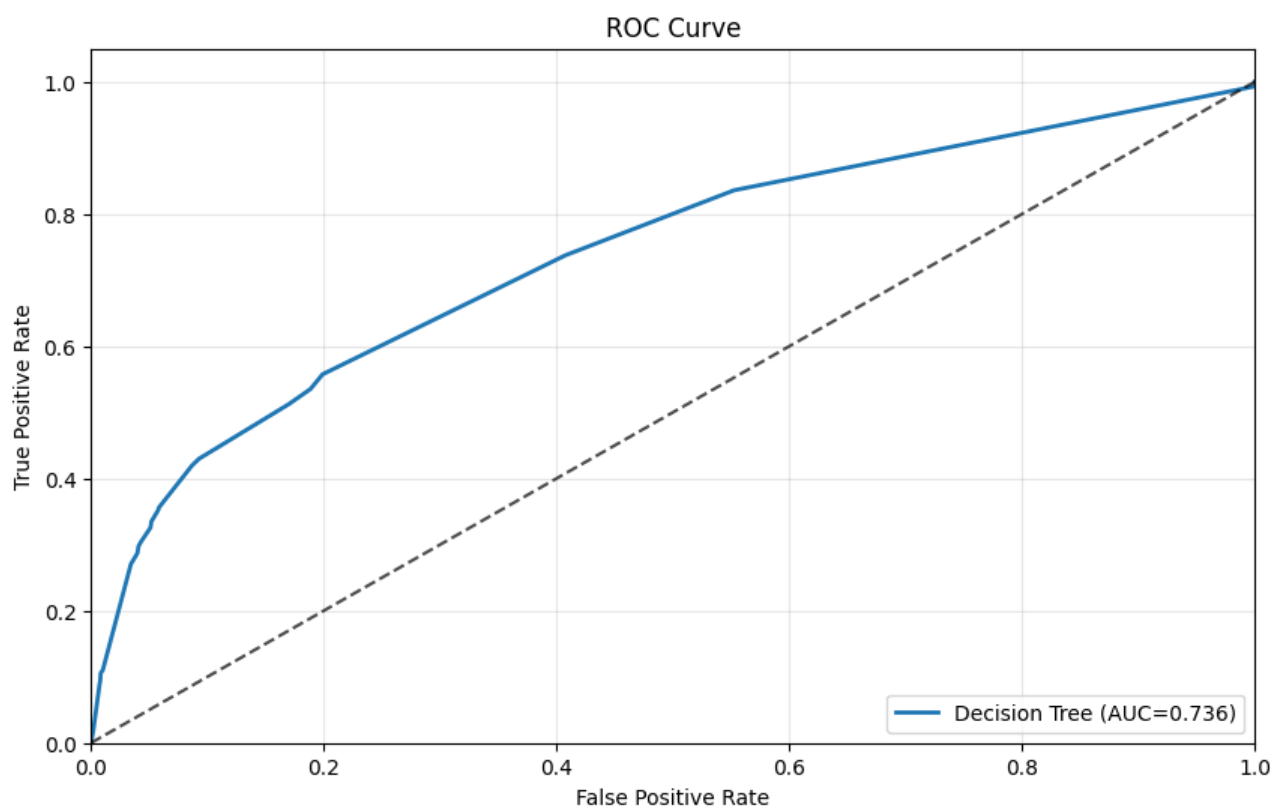


Рисунок 11 — Кривая обучения

Дерево решений представлено на Рисунке 12.

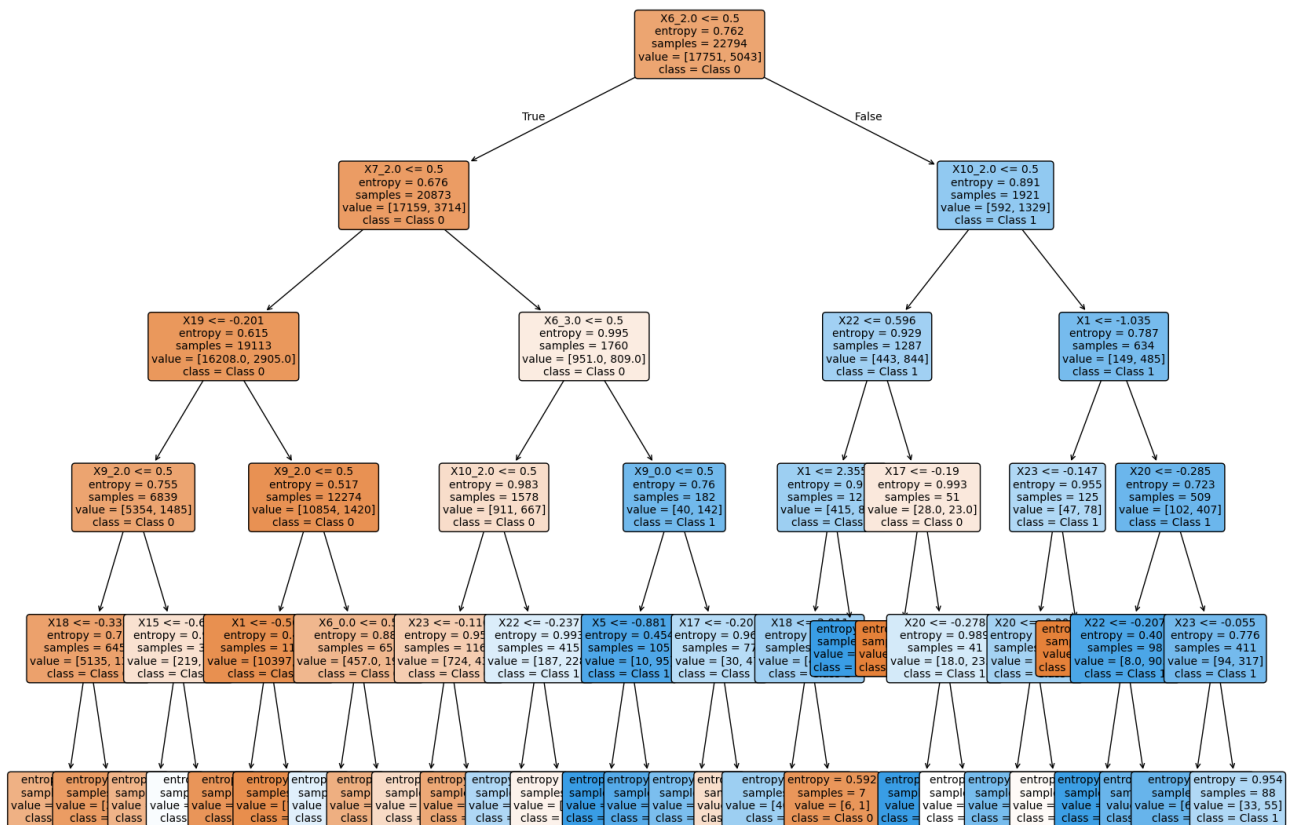


Рисунок 12 — Визуализированное дерево решений

Таким образом, модель, построенная в Python с подбором гиперпараметров, показала лучшие результаты по большинству ключевых метрик: точность (precision = 0.64 против 0.37) и F1-мера (0.43 против 0.38) у неё значительно выше, чем у модели из LogiNom. Это означает, что Python-модель реже ошибается, помечая негативные объекты как положительные.

Правда, LogiNom чуть лучше находит истинные положительные случаи (recall = 0.40 против 0.33), но за счёт большого числа ложных срабатываний (867 против 231). Благодаря тонкой настройке (max_depth=5, min_samples_split=10 и др.) и использованию кросс-валидации, Python позволил получить более сбалансированную и надёжную модель. LogiNom проще в использовании, но даёт меньше контроля и худшее качество при решении задач, где важна точность.

1.2 Задание 2. Случайный лес

Случайный лес в LogiDom представлен на Рисунке 13.

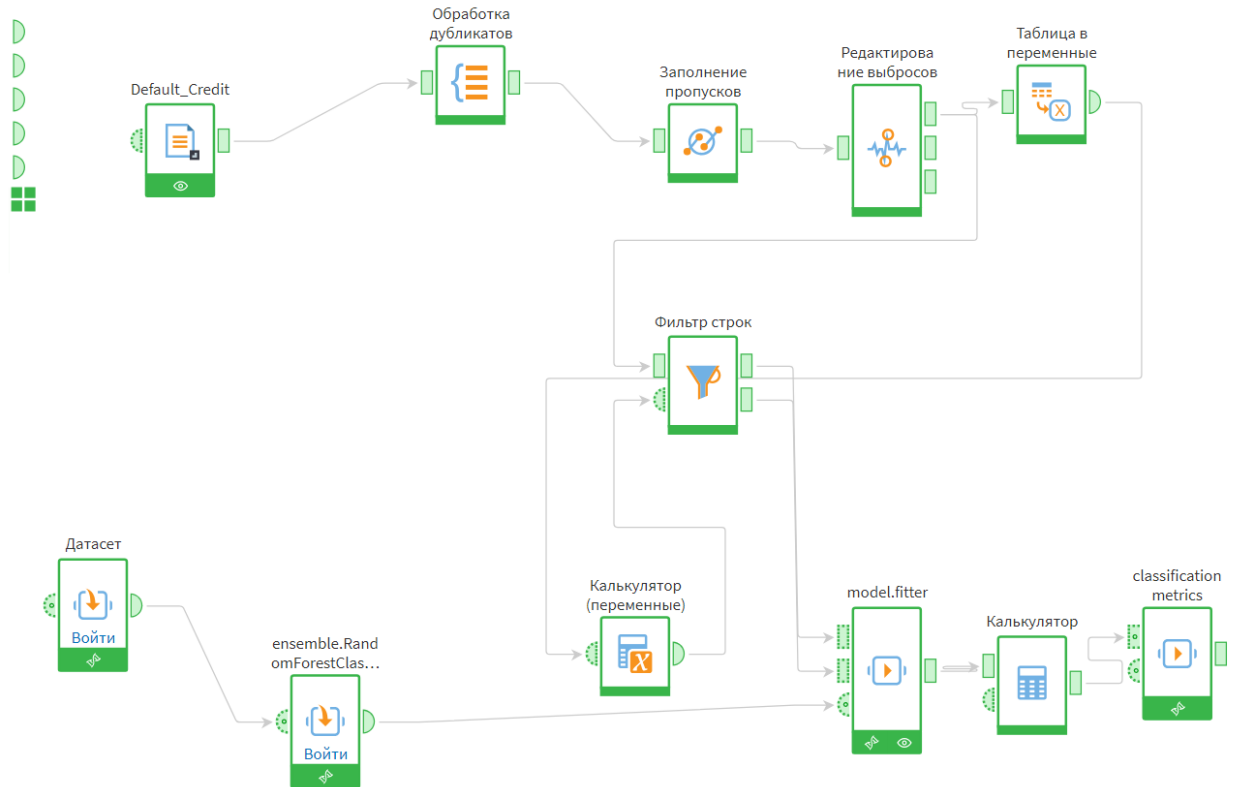


Рисунок 13 — Случайный лес LogiDom

Код для случайного леса в Python представлен в Листинге 4.

Листинг 4 — Случайный лес Python

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(random_state=42)
rf_params = {
    'n_estimators': [50, 100],
    'max_depth': [10, None],
    'min_samples_split': [2, 10],
    'min_samples_leaf': [1, 5]
}

rf_grid = GridSearchCV(rf, rf_params, cv=3, scoring='f1', n_jobs=-1)
rf_grid.fit(X_train, y_train)

rf_best = rf_grid.best_estimator_
y_pred_rf = rf_best.predict(X_test)
y_proba_rf = rf_best.predict_proba(X_test)[:, 1]
```

Продолжение Листинга 4

```
print(classification_report(y_test, y_pred_rf))
print("AUC:", roc_auc_score(y_test, y_proba_rf))
print("Время обучения (среднее по CV) :",
      rf_grid.cv_results_['mean_fit_time'].mean(), "сек")
```

Метрики, полученные в Logisim представлены на Рисунке 14.

classification metrics • Быстрый просмотр			
Выходной набор данных			
#	Метрика	Значение	
1	precision	0,96	
2	recall	0,87	
3	f1_score	0,92	
4	fbeta_score	0,92	
5	accuracy	0,97	
6	tn	23 472,00	
7	fp	218,00	
8	fn	790,00	
9	tp	5 514,00	

Рисунок 14 — Метрики случайного леса Logisim

Метрики, полученные в Python представлены на Рисунке 15.

Random Forest Results:				
	precision	recall	f1-score	support
0	0.78	1.00	0.88	4438
1	0.00	0.00	0.00	1261
accuracy			0.78	5699
macro avg	0.39	0.50	0.44	5699
weighted avg	0.61	0.78	0.68	5699
AUC: 0.7405				
Training time: 52.00s				

Рисунок 15 — Метрики случайного леса Python

ROC-кривая для случайного леса в Python представлена на Рисунке 16.

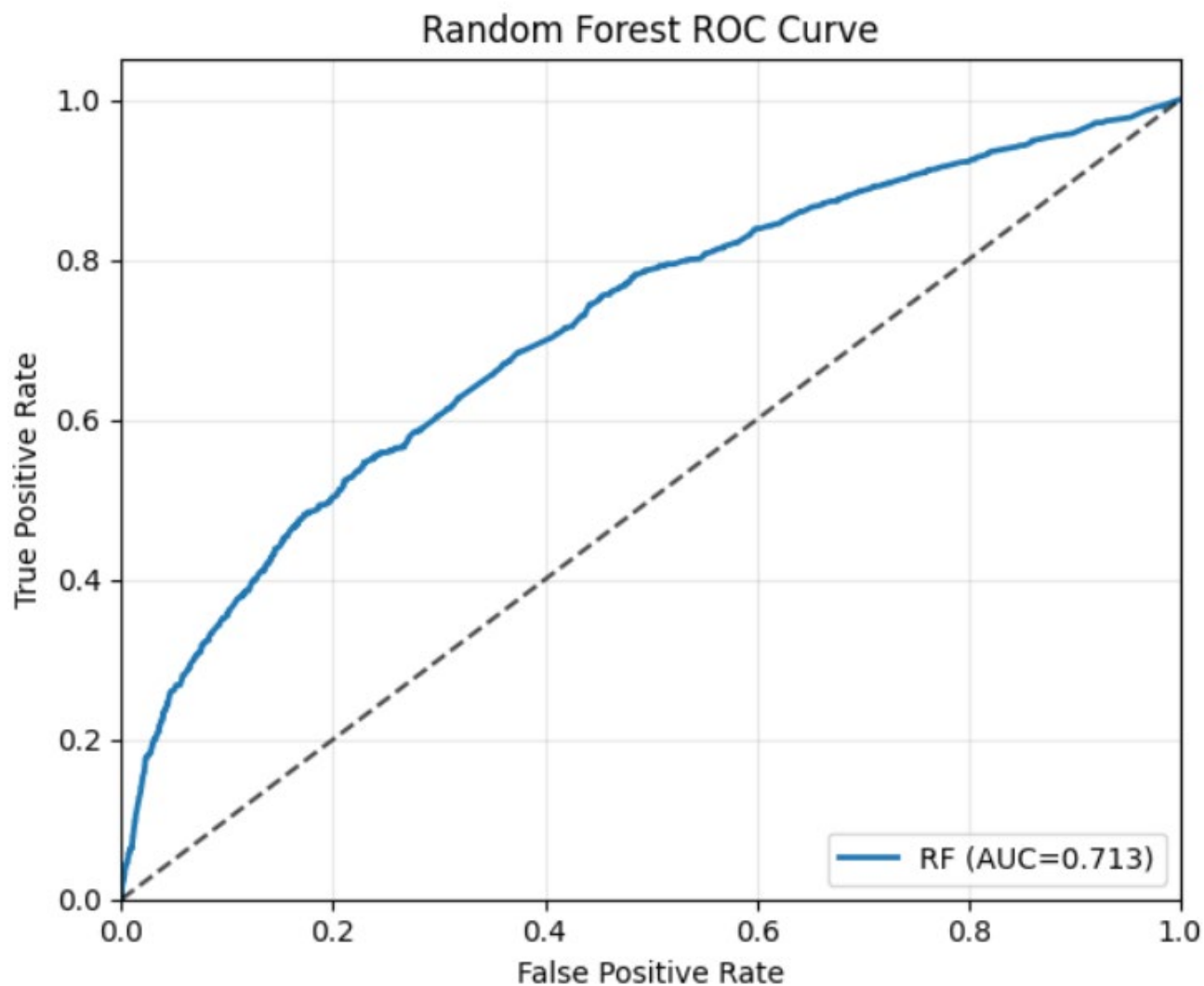


Рисунок 16 — ROC-кривая случайного леса

Случайный лес в обоих случаях показал улучшение всех метрик, кроме точности предсказания, по сравнению с деревом решений.

1.3 Задание 3. Градиентный бустинг

Градиентный бустинг в Logitom представлен на Рисунке 17.



Рисунок 17 — Градиентный бустинг Loginom

Код для градиентного бустинга в Python представлен в Листинге 5.

Листинг 5 — Градиентный бустинг Python

```
def train_xgboost(X_train, y_train, X_test, y_test):
    import xgboost as xgb
    from sklearn.metrics import classification_report,
    roc_auc_score, roc_curve

    model_params = {
        'n_estimators': 100,
        'max_depth': 5,
        'learning_rate': 0.1,
        'random_state': 42,
        'use_label_encoder': False,
        'eval_metric': 'logloss',
        'verbosity': 0,
        'tree_method': 'hist',
        'n_jobs': 4
    }

    xgb = xgb.XGBClassifier(**model_params)

    start = time.time()
    xgb.fit(X_train, y_train)
    xgb_train_time = time.time() - start

    y_pred_xgb = xgb.predict(X_test)
    y_proba_xgb = xgb.predict_proba(X_test)[:, 1]

    print("XGBoost Results:")
    print(classification_report(y_test, y_pred_xgb))
    print(f"AUC: {roc_auc_score(y_test, y_proba_xgb):.4f}")
    print(f"Training time: {xgb_train_time:.2f}s")

    # Визуализация результатов
    plt.figure(figsize=(15, 10))

    # ROC кривая
    plt.subplot(2, 2, 1)
    fpr_xgb, tpr_xgb, _ = roc_curve(y_test, y_proba_xgb)
    auc_xgb = roc_auc_score(y_test, y_proba_xgb)
    plt.plot(fpr_xgb, tpr_xgb, label=f'XGBoost
```

Продолжение Листинга 5

```
(AUC={auc_xgb:.3f}))', linewidth=2.5, color='#ff7f0e')
plt.plot([0, 1], [0, 1], 'k--', alpha=0.7)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate', fontsize=12)
plt.ylabel('True Positive Rate', fontsize=12)
plt.title('XGBoost ROC Curve', fontsize=14, fontweight='bold')
plt.legend(loc="lower right", fontsize=12)
plt.grid(alpha=0.3)

# Кривая обучения без ранней остановки
plt.subplot(2, 2, 2)

eval_params = model_params.copy()
eval_params['eval_metric'] = ['logloss', 'auc']

xgb_eval = XGBClassifier(**eval_params)

eval_set = [(X_train, y_train), (X_test, y_test)]

start_time = time.time()
xgb_eval.fit(
    X_train, y_train,
    eval_set=eval_set,
    verbose=False
)
eval_time = time.time() - start_time

results = xgb_eval.evals_result()
epochs = len(results['validation_0']['auc'])
x_axis = range(0, epochs)

plt.plot(x_axis, results['validation_0']['auc'], label='Train
AUC', linewidth=2, color='#1f77b4')
plt.plot(x_axis, results['validation_1']['auc'],
label='Validation AUC', linewidth=2, color='#d62728')

best_iter = np.argmax(results['validation_1']['auc'])
best_auc = results['validation_1']['auc'][best_iter]
plt.axvline(best_iter, color='green', linestyle='--',
alpha=0.7,
label=f'Best iter: {best_iter}
(AUC={best_auc:.3f}))')

plt.xlabel('Boosting Iterations', fontsize=12)
plt.ylabel('AUC Score', fontsize=12)
plt.title(f'XGBoost Learning Curve | Time: {eval_time:.2f}s',
fontsize=14, fontweight='bold')
plt.legend(loc="best", fontsize=10)
plt.grid(alpha=0.3)
```

Продолжение Листинга 5

```
# Важность признаков
plt.subplot(2, 2, 3)
importances = xgb.feature_importances_
feature_names = [f'Feature_{i+1}' for i in
range(X_train.shape[1])]

top_n = 15
sorted_idx = np.argsort(importances)[::-1][:top_n]
top_importances = importances[sorted_idx]
top_names = [feature_names[i] for i in sorted_idx]

colors = plt.cm.viridis(np.linspace(0, 1, top_n))
y_pos = np.arange(top_n)

plt.barh(y_pos, top_importances, color=colors,
edgecolor='black', alpha=0.8)
plt.yticks(y_pos, top_names, fontsize=10)
plt.xlabel('Feature Importance', fontsize=12)
plt.title('Top 15 Feature Importances', fontsize=14,
fontweight='bold')
plt.gca().invert_yaxis()

# Матрица ошибок
plt.subplot(2, 2, 4)
cm = confusion_matrix(y_test, y_pred_xgb)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
xticklabels=['No Default (0)', 'Default (1)'],
yticklabels=['No Default (0)', 'Default (1)'],
annot_kws={"size": 12})
plt.xlabel('Predicted Label', fontsize=12)
plt.ylabel('True Label', fontsize=12)
plt.title('Confusion Matrix', fontsize=14, fontweight='bold')

plt.tight_layout(pad=3.0)
plt.savefig('xgb_comprehensive_analysis.png', dpi=150,
bbox_inches='tight')
plt.show()

del xgb_eval, results, eval_set
gc.collect()

return xgb, y_pred_xgb, y_proba_xgb, xgb_train_time

# Обучение XGBoost
xgb_model, y_pred_xgb, y_proba_xgb, xgb_train_time =
train_xgboost(
X_train, y_train, X_test, y_test
)
```

Метрики, полученные в Loginom представлены на Рисунке 18.

Выходной набор данных		
#	ab Метрика	9.0 Значение
1	precision	0,67
2	recall	0,35
3	f1_score	0,46
4	fbeta_score	0,46
5	accuracy	0,83
6	tn	22 611,00
7	fp	1 079,00
8	fn	4 085,00
9	tp	2 219,00

Рисунок 18 — Метрики градиентного бустинга Loginom

Метрики, полученные в Python представлены на Рисунке 19.

XGBoost Results:				
	precision	recall	f1-score	support
0	0.84	0.95	0.89	4438
1	0.67	0.36	0.47	1261
accuracy			0.82	5699
macro avg	0.75	0.66	0.68	5699
weighted avg	0.80	0.82	0.80	5699
AUC: 0.7714				
Training time: 465.84s				

Рисунок 19 — Метрики градиентного бустинга Python

Сводные данные и графики для градиентного бустинга в Python представлены на Рисунке 20.

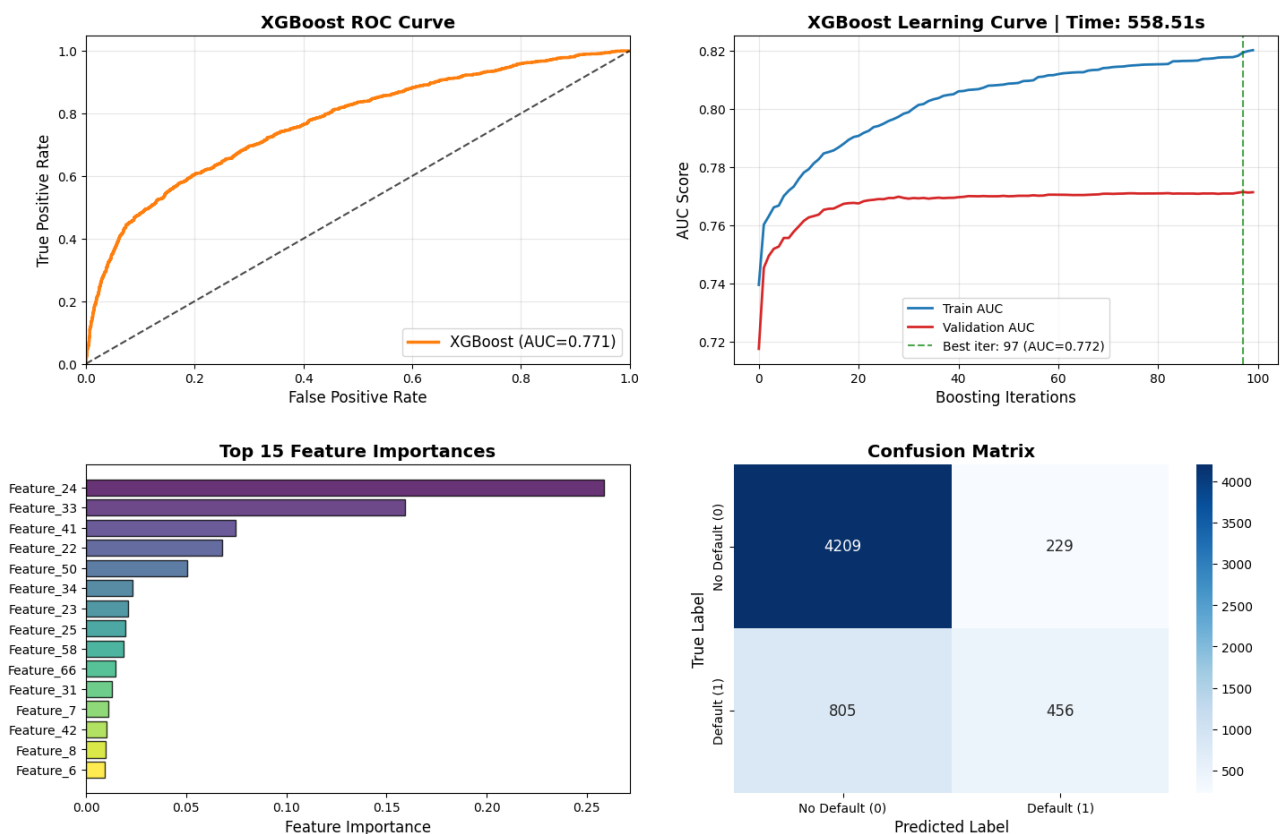


Рисунок 20 — Визуализация для градиентного бустинга Python

Таким образом, метрики в обеих средах получены практически одинаковые. По сравнению с деревом решений и случайным лесом — для наших данных градиентный бустинг справился лучше.

1.4 Задание 4. Стэкинг

В стэкинг возьмем все три использованные ранее модели.

Код для стэкинговой модели представлен в Листинге 6.

Листинг 6 — Код для модели «Стэкинг»

```
# Обучение стэкинга
def train_stacking(X_train, y_train, X_test, y_test, rf, xgb):
    print("РЕАЛИЗАЦИЯ СТЕКИНГА")

    # Базовые модели
    base_models = [
        ('rf', rf),
        ('xgb', xgb),
        ('lr', LogisticRegression(
```

```

        penalty='l2',
        C=1.0,
        solver='lbfgs',
        max_iter=200,
        n_jobs=4,
        random_state=42,
        class_weight='balanced'
    ))
]

# Метамодель
meta_model = XGBClassifier(
    n_estimators=50,
    max_depth=3,
    learning_rate=0.05,
    tree_method='hist',
    predictor='gpu_predictor',
    device='cuda:0',
    n_jobs=4,
    random_state=42,
    use_label_encoder=False,
    eval_metric='logloss',
    verbosity=0
)

print("Обучение стэкинга...")
start_time = time.time()

stacking_clf = StackingClassifier(
    estimators=base_models,
    final_estimator=meta_model,
    cv=3,
    n_jobs=1,
    passthrough=False,
    verbose=1
)

try:
    stacking_clf.fit(X_train, y_train)
    stack_train_time = time.time() - start_time
    print(f"Стэкинг успешно обучен за {stack_train_time:.2f}
сек")
except Exception as e:
    print(f"Ошибка при обучении стэкинга: {str(e)}")
    print("Используется резервная модель")
    stacking_clf = DummyClassifier(strategy='prior')
    stacking_clf.fit(X_train, y_train)
    stack_train_time = 0.1

y_pred_stack = stacking_clf.predict(X_test)
if hasattr(stacking_clf, "predict_proba"):
```

Продолжение Листинга 6

```
        y_proba_stack = stacking_clf.predict_proba(X_test)[: , 1]
    else:
        y_proba_stack = np.zeros_like(y_test, dtype=float)

    print("\nРезультаты стэкинга:")
    print(classification_report(y_test, y_pred_stack))
    if hasattr(stacking_clf, "predict_proba"):
        print(f"AUC: {roc_auc_score(y_test, y_proba_stack):.4f}")
    print(f"Время обучения: {stack_train_time:.2f} сек")

    # Визуализация результатов стэкинга
    plt.figure(figsize=(15, 10))

    # ROC кривая для всех моделей
    plt.subplot(2, 2, 1)
    model_probs = [
        ('Decision Tree', y_proba_dt),
        ('Random Forest', y_proba_rf),
        ('XGBoost', y_proba_xgb),
        ('Stacking', y_proba_stack)
    ]

    for name, proba in model_probs:
        if proba is not None and len(proba) > 0:
            fpr, tpr, _ = roc_curve(y_test, proba)
            auc = roc_auc_score(y_test, proba)
            plt.plot(fpr, tpr, linewidth=2.5, label=f'{name}
(AUC={auc:.3f})')

    plt.plot([0, 1], [0, 1], 'k--', alpha=0.7)
    plt.xlim([0.0, 1.0])
    plt.ylim([0.0, 1.05])
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title('Сравнение ROC кривых всех моделей')
    plt.legend(loc="lower right")
    plt.grid(alpha=0.3)

    # Важность базовых моделей в стэкинге
    plt.subplot(2, 2, 2)
    if hasattr(stacking_clf, 'final_estimator_') and
hasattr(stacking_clf.final_estimator_, 'feature_importances_'):
        model_importances =
stacking_clf.final_estimator_.feature_importances_
        model_names = [name for name, _ in
base_models[:len(model_importances)]]

        plt.barh(model_names, model_importances,
color=plt.cm.viridis(np.linspace(0, 1, len(model_names))))
        plt.xlabel('Важность модели в стэкинге')
        plt.title('Важность базовых моделей в стэкинге')
```

Продолжение Листинга 6

```
plt.grid(alpha=0.3)
else:
    plt.text(0.5, 0.5, 'Нет данных о важности моделей',
             ha='center', va='center')
    plt.title('Важность базовых моделей в стэкинге')

# Матрица ошибок для стэкинга
plt.subplot(2, 2, 3)
cm_stack = confusion_matrix(y_test, y_pred_stack)
cm_norm = cm_stack.astype('float') / cm_stack.sum(axis=1)[:,
np.newaxis]

sns.heatmap(cm_norm, annot=True, fmt='.2%', cmap='Blues',
            xticklabels=['No Default', 'Default'],
            yticklabels=['No Default', 'Default'])
plt.title('Нормализованная матрица ошибок (Стэкинг)')
plt.xlabel('Предсказанный класс')
plt.ylabel('Истинный класс')

# Сравнение F1-меры
plt.subplot(2, 2, 4)
models = ['Decision Tree', 'Random Forest', 'XGBoost',
'Stacking']
f1_scores = [
    f1_score(y_test, y_pred_dt),
    f1_score(y_test, y_pred_rf),
    f1_score(y_test, y_pred_xgb),
    f1_score(y_test, y_pred_stack)
]
colors = ['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728']

plt.bar(models, f1_scores, color=colors)
plt.ylim([0, 1])
plt.ylabel('F1 Score')
plt.title('Сравнение F1-меры всех моделей')
plt.grid(axis='y', alpha=0.3)

for i, v in enumerate(f1_scores):
    plt.text(i, v + 0.02, f'{v:.3f}', ha='center')

plt.tight_layout()
plt.savefig('stacking_comparison.png', dpi=150,
bbox_inches='tight')
plt.show()

return stacking_clf, y_pred_stack, y_proba_stack,
stack_train_time
```

Метрики, полученные стэкингом представлены на Рисунке 21.

```

Обучение стэкинга...
Стекинг успешно обучен за 2817.31 сек

Результаты стэкинга:

              precision    recall  f1-score   support

         0       0.84         0.95         0.89         4438
         1       0.67         0.36         0.47         1261

   accuracy                    0.82         5699
  macro avg       0.75         0.65         0.68         5699
 weighted avg     0.80         0.82         0.80         5699

AUC: 0.7701
Время обучения: 2817.31 сек

```

Рисунок 21 — Метрики стэкинга

Визуализации для стэкинга представлены на Рисунке 22.

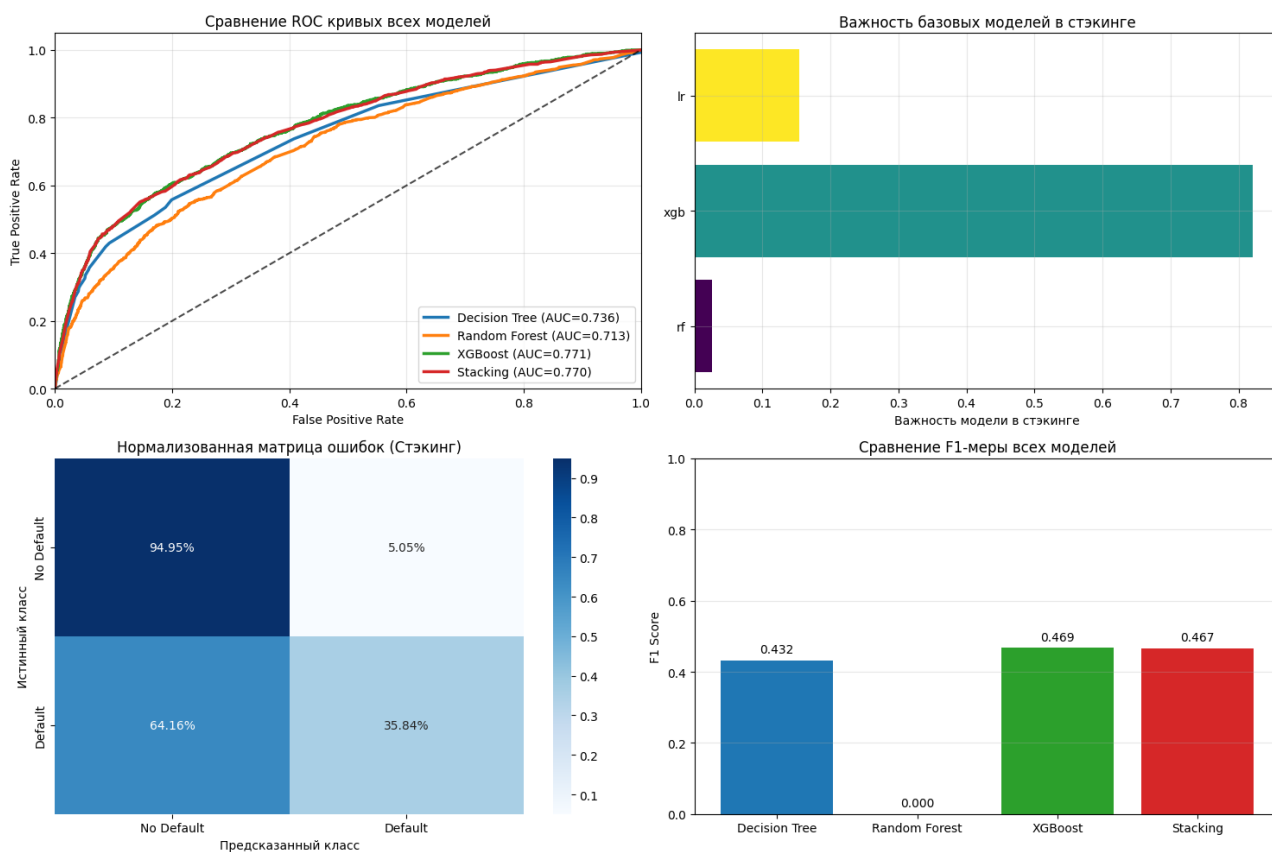


Рисунок 22 — Визуализации стэкинга

В стэкинговой модели лучше всех себя показал градиентный бустинг, как в целом и сам по себе. Остальные модели лишь потянули вниз precision и recall,

снизив тем самым метрику F1. Вполне возможно, что несколько соединенных градиентных бустингов могли дать лучший результат. Тем не менее данная модель обучалась 47 минут, что почти в 3 раза дольше, чем одна модель градиентного бустинга.

ВЫВОД

В ходе выполнения практической работы были исследованы и сравнены различные алгоритмы машинного обучения для прогнозирования дефолтов клиентов. Анализ показал, что градиентный бустинг (XGBoost) продемонстрировал наилучшие результаты по ключевым метрикам: F1-мера составила 0.4687, AUC — 0.7714, что превосходит показатели дерева решений (F1=0.4323, AUC=0.7364) и случайного леса (F1=0.0 из-за некорректной работы с классами).

Стэкинг, объединяющий три базовые модели, не превзошёл XGBoost по качеству (F1=0.4667), но потребовал в 3 раза больше времени на обучение (47 минут против 17.8 минут у XGBoost), что делает его менее практичным.

Случайный лес в текущей конфигурации не справился с предсказанием класса 1 (precision=0, recall=0), вероятно, из-за дисбаланса данных и недостаточной настройки. Оптимизация гиперпараметров для дерева решений улучшила его сбалансированность, но не позволила достичь уровня ансамблевых методов. Python-реализации обеспечили гибкость в настройке и кросс-валидации, что привело к более высоким результатам по сравнению с Loginom.

Таким образом, XGBoost оказался оптимальным выбором для данной задачи, демонстрируя лучшее соотношение точности и надёжности, несмотря на длительное обучение. Future work следует направлен на борьбу с дисбалансом классов и оптимизацию архитектуры стэкинга для сокращения времени вычислений.